

PRACTICA 1: PIPELINES DE DATOS CON AIRFLOW

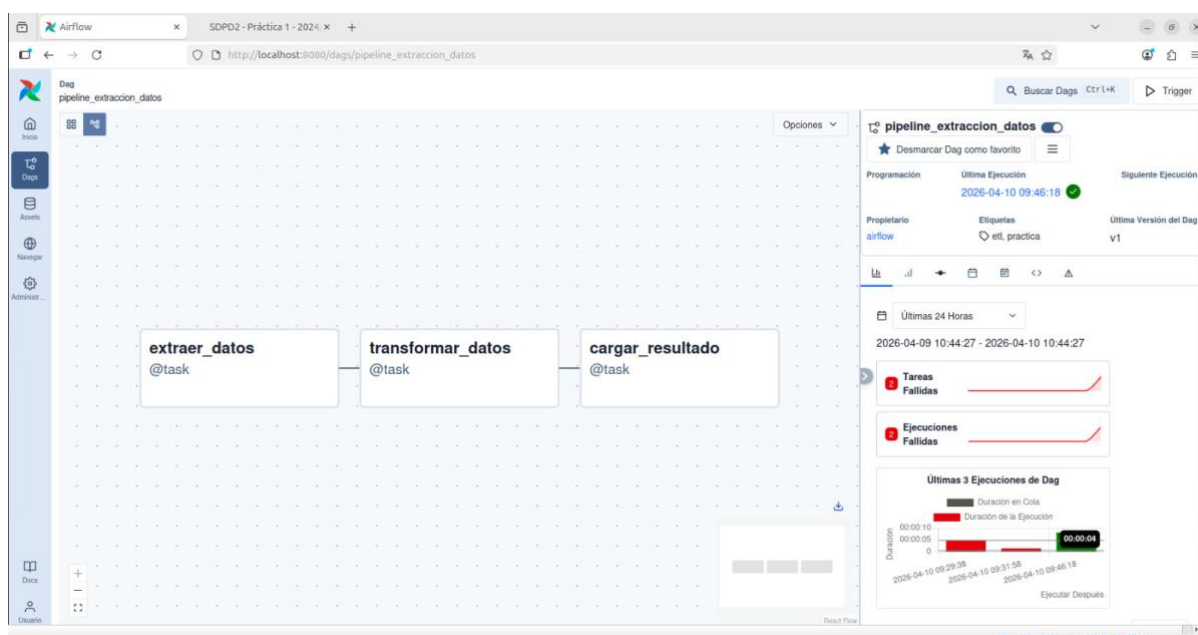
GRUPO 2: FRANCISCO JAVIER MARTÍN FERNÁNDEZ, LARA MONTERO DEL PRADO, MÓNICA
SILVA DORADO Y CELIA LÓPEZ GARCÍA

Introducción

Para esta práctica se ha trabajado con Apache Airflow con el objetivo de diseñar distintos DAGs relacionados con el procesamiento de datos. El trabajo se ha centrado en varias fases del pipeline, siguiendo una lógica general de tipo ETL, es decir, extracción, preparación o limpieza, transformación y carga final del resultado.

A continuación, se describe la estructura y funcionamiento de cada uno de los DAGs implementados.

1. DAG de extracción: pipeline_extraccion_datos



El primer DAG del trabajo es `pipeline_extraccion_datos`, y representa la fase inicial del pipeline. Su finalidad es obtener el dataset original y dejar preparado el resultado de esta primera etapa.

En la captura del grafo se observan tres tareas: `extraer_datos`, `transformar_datos` y `cargar_resultado`. Estas tareas aparecen organizadas de forma secuencial, lo que indica que cada una depende de la anterior y que el flujo de ejecución sigue un orden lineal.

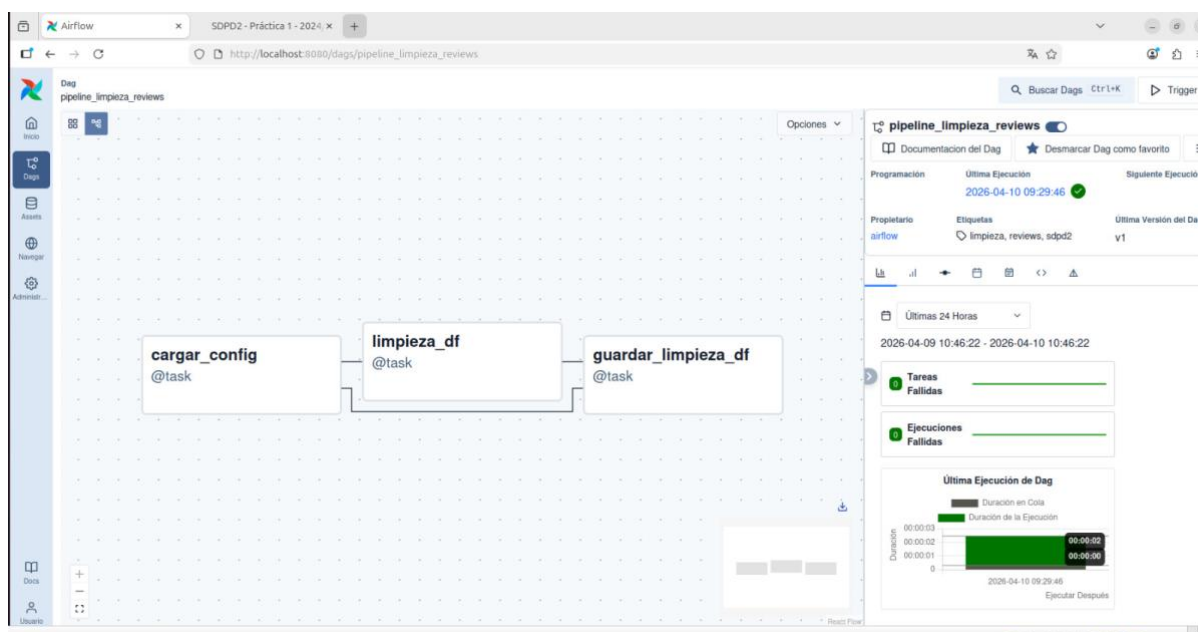
La tarea `extraer_datos` es la primera en ejecutarse. Su propósito es acceder a la fuente de datos definida previamente y obtener el archivo de entrada. Esta fase es fundamental, ya que constituye el punto de partida del resto del pipeline. Sin la extracción del archivo en bruto no sería posible continuar con los siguientes pasos.

A continuación, se ejecuta la tarea `transformar_datos`. Aunque por el nombre podría parecer una transformación completa, dentro de este DAG actúa más bien como una fase intermedia de preparación del resultado de extracción. Su función es servir de enlace entre la obtención del fichero y la carga del resultado, manteniendo así una estructura ordenada en el flujo del DAG.

Por último, la tarea `cargar_resultado` cierra esta fase del proceso. En este contexto, su función es dejar el resultado de la extracción listo para que pueda ser utilizado posteriormente. No se trata todavía de la carga definitiva en Kafka, sino del cierre de esta primera etapa.

En cuanto al grafo, la estructura es sencilla y fácil de interpretar. Se trata de un flujo lineal de tres pasos consecutivos, lo cual tiene bastante sentido en una fase inicial como esta. Además, en la captura se observa que hubo algunas ejecuciones fallidas anteriores, aunque la última ejecución aparece completada correctamente. Esto refleja el proceso normal de desarrollo y prueba del DAG hasta conseguir que funcionara de manera adecuada.

2. DAG de limpieza: `pipeline_limpieza_reviews`



El segundo DAG implementado es `pipeline_limpieza_reviews`, que corresponde a la fase de limpieza y preparación de los datos. Esta parte del trabajo es especialmente importante, ya que antes de transformar o cargar la información es necesario asegurarse de que los datos tengan una calidad mínima y una estructura coherente.

En el grafo se observan tres tareas: `cargar_config`, `limpieza_df` y `guardar_limpieza_df`.

La tarea `cargar_config` es la encargada de leer el archivo `config.toml`, donde se encuentran definidas las rutas de entrada y salida que utiliza el pipeline. Esto se ajusta a uno de los requisitos del enunciado, que indica que las variables de configuración deben estar separadas del código y codificadas en un fichero `.toml`. Esta tarea resulta útil porque evita dejar rutas fijas dentro del programa y hace que el código quede más ordenado y fácil de modificar.

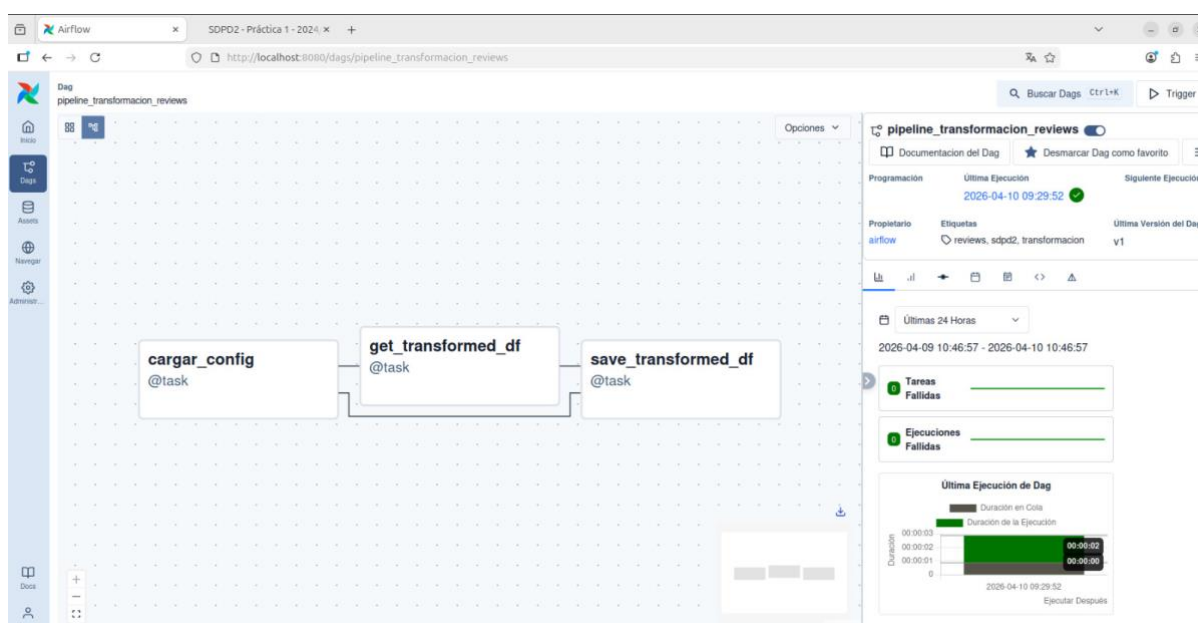
La tarea principal de este DAG es `limpieza_df`. En ella se lee el archivo de datos en bruto y se aplican distintas operaciones de limpieza. En nuestro caso, se eliminan duplicados, se eliminan filas con valores nulos, se corrigen espacios sobrantes en columnas de texto y se normalizan ciertos campos. También se convierten identificadores a formato texto, con el objetivo de evitar problemas posteriores

al tratarlos como variables numéricas, y se transforma la columna de fecha a un tipo adecuado para su uso posterior. Esta fase encaja con lo que el enunciado plantea sobre limpieza de datos, como la homogeneización de valores, detección de datos faltantes o comprobación de tipos.

La última tarea, `guardar_limpieza_df`, recibe el resultado limpio generado anteriormente y lo guarda en un nuevo archivo CSV. De esta forma, el resultado de la fase de limpieza queda almacenado y preparado para servir como entrada a la fase de transformación.

La secuencia del grafo es clara y lógica. Primero se carga la configuración, luego se limpian los datos y finalmente se guardan. Además, en la captura se observa que la ejecución se ha completado correctamente, sin tareas fallidas, lo que indica que esta parte del pipeline ha funcionado de forma adecuada.

3. DAG de transformación: `pipeline_transformacion_reviews`



El tercer DAG del trabajo es `pipeline_transformacion_reviews`, que corresponde a la fase de transformación de los datos. En esta etapa ya no se trata solo de corregir problemas básicos, sino de generar variables nuevas y dejar el dataset en un formato más útil para su uso posterior.

En la captura del grafo aparecen tres tareas: `cargar_config`, `get_transformed_df` y `save_transformed_df`.

La primera tarea, `cargar_config`, vuelve a encargarse de leer el archivo `config.toml`. Igual que en el DAG de limpieza, esto permite mantener separada la configuración del código principal y utilizar rutas y parámetros de manera centralizada. Así, el flujo mantiene una estructura más ordenada y coherente.

La tarea más importante de esta fase es `get_transformed_df`. Su función es leer el archivo limpio generado anteriormente y aplicar varias transformaciones sobre él. En nuestro caso, se ha trabajado

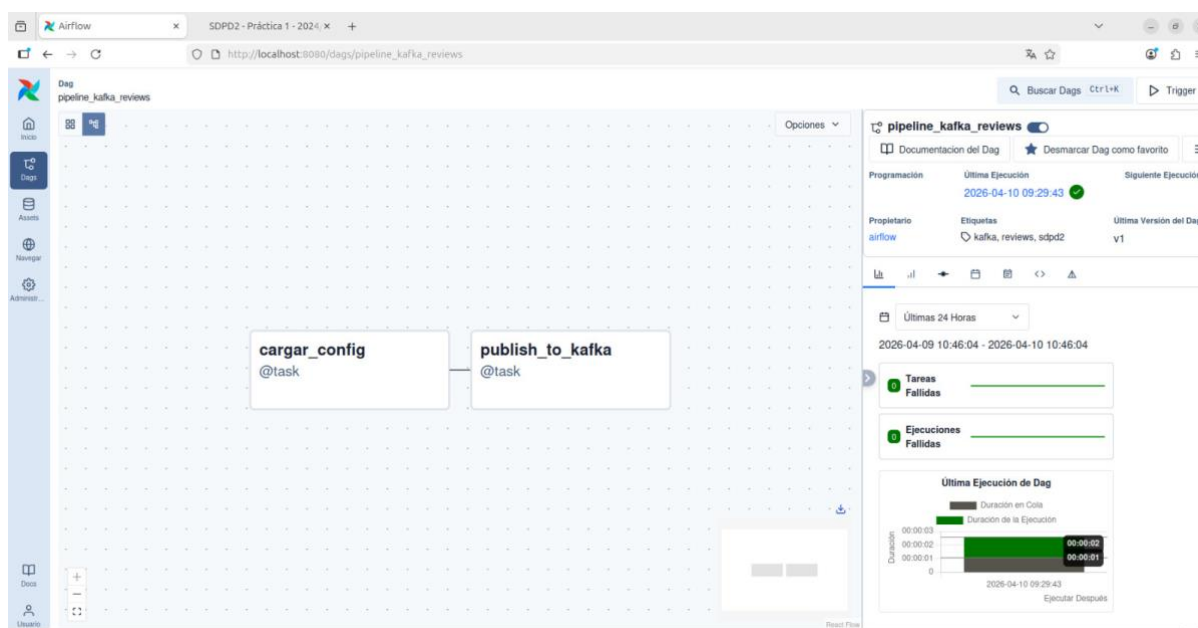
especialmente con la columna `comments`, ya que era una de las más representativas del dataset. A partir de ella se genera una nueva columna llamada `comments_clean`, en la que los comentarios se pasan a minúsculas, se eliminan espacios repetidos y se quitan signos de puntuación para dejar el texto en un formato más simple.

Además, en esta misma tarea se crean nuevas variables derivadas. Entre ellas destacan `comment_length`, que mide la longitud del comentario, y `word_count`, que indica el número de palabras de cada reseña. También se aprovecha la columna `date` para obtener nuevas variables temporales, concretamente `review_year` y `review_month`, lo que permite trabajar de una forma más sencilla con los datos.

La última tarea, `save_transformed_df`, guarda el resultado transformado en un nuevo archivo CSV. Este archivo es el que se utilizará después como entrada para la fase final de publicación en Kafka.

En cuanto a la estructura del grafo, se repite un patrón muy parecido al del DAG de limpieza: primero se carga la configuración, después se aplican las transformaciones y finalmente se guarda el resultado. La secuencia es clara y fácil de seguir, y además la captura muestra que la ejecución se ha completado correctamente y sin errores.

4. DAG de publicación en Kafka: `pipeline_kafka_reviews`



El último DAG implementado es `pipeline_kafka_reviews`, correspondiente a la fase final del trabajo. En este caso, el objetivo es publicar el resultado final del pipeline en Apache Kafka, tal y como pide el enunciado.

En la captura del grafo aparecen dos tareas: `cargar_config` y `publish_to_kafka`.

La tarea `cargar_config` vuelve a leer el archivo `config.toml`, pero en este caso no solo obtiene la ruta del archivo transformado, sino también la configuración relacionada con Kafka, como el `topic` y la dirección del servidor `bootstrap`. Esta tarea es necesaria para que el DAG se pueda conectar correctamente al sistema de mensajería.

Después se ejecuta la tarea `publish_to_kafka`, que es la parte principal de esta fase. En ella se abre el archivo CSV transformado y se recorre fila por fila. Cada fila se convierte en un mensaje con formato JSON y se envía al `topic` de Kafka definido en la configuración. De este modo, los datos ya preparados quedan listos para ser utilizados en la siguiente fase de procesamiento.

Este DAG tiene una estructura más corta y directa que los anteriores, ya que su finalidad es muy concreta. Primero se carga la configuración y después se realiza la publicación. A pesar de ser más simple, este DAG representa la carga final del resultado y da continuidad al pipeline en un entorno de procesamiento distribuido.

En la captura se observa que la ejecución se ha completado correctamente y que no hay tareas fallidas, lo que indica que la conexión con Kafka y el envío de mensajes se han realizado de forma satisfactoria.

Valoración final del trabajo

Si se observa el trabajo en conjunto, la secuencia ha sido la siguiente. En primer lugar, se ha desarrollado una fase de extracción orientada a obtener los datos en bruto. A continuación, se ha implementado una fase de limpieza, en la que se corrigen problemas básicos del dataset y se genera un archivo más fiable. Después, se ha llevado a cabo una fase de transformación, en la que se crean nuevas variables y se mejora el formato de algunas columnas, especialmente las textuales. Finalmente, se ha implementado una fase de publicación en Kafka, donde el resultado final se envía a un `topic` para continuar el procesamiento en etapas posteriores.

En general, el trabajo realizado permite representar de manera bastante clara las distintas etapas de un pipeline de datos con Airflow. Una de las partes más útiles ha sido poder visualizar la secuencia de tareas mediante los grafos, ya que esto ayuda a entender mejor el flujo de ejecución y también facilita la detección de errores durante el desarrollo.

Además, el uso de un fichero `config.toml` ha permitido mantener separada la configuración del código, lo que hace que la solución quede más ordenada.

En conclusión, el conjunto del trabajo sigue una lógica progresiva de preparación de datos y cumple con los requisitos principales de la práctica: uso de Airflow con tareas definidas de forma clara, secuencia de ejecución visible en los grafos, configuración separada en `.toml`, tratamiento de los datos en varias etapas y publicación final del resultado en Apache Kafka.